P2539R0
Should the output of std::print to a terminal be synchronized with the underlying stream?

Published Proposal, 2022-02-05

Author:
Victor Zverovich

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

§ 1. Discussion

To prevent mojibake `std::print` may use a native Unicode API when writing to a terminal bypassing the stream buffer. During the review of [P2093] "Formatted output" Tim Song suggested that synchronizing `std::print` with the underlying stream may be beneficial for gradual adoption. This paper briefly discusses this option.

Consider the following example:

```
printf("first\n");
std::print("second\n");
```

This will produce the expected output:

```
first
second
```

because `stdout` is line buffered by default.

However, this may reorder the output:

```
printf("first");
std::print("second");
```

because of buffering in `printf` but not `std::print`.

It is possible to prevent reordering by flushing the buffer before writing to a terminal in `std::print` making this case less surprising. This will incur additional cost but only for the terminal case and when transcoding is needed.

Neither {fmt} ([FMT]) nor Rust ([RUST-STDIO]) do such synchronization in their implementations of `print`.

2. References

References

Informative References

[FMT]
Victor Zverovich; et al. The fmt library. URL: https://github.com/fmtlib/fmt

[P2093]
Victor Zverovich. Formatted output. URL: https://wg21.link/p2093

[RUST-STDIO]

The Rust Programming Language repository, windows_stdio. URL: https://github.com/rust-lang/rust/blob/db492ecd5ba6bd82205612cebb9034710653f0c2/library/std/src/sys/windows/stdio.rs